SISTEMAS DISTRIBUÍDOS

CMP 1190

Me. Ricardo de Andrade Kratz

PUC GOIÁS

Aula de Revisão

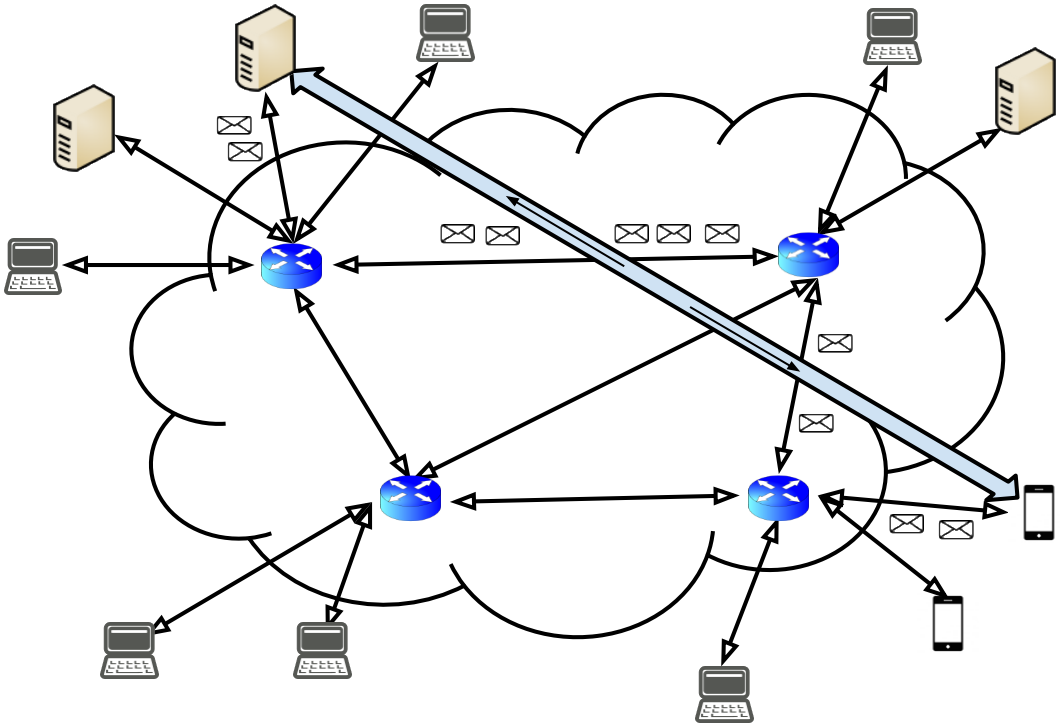
Socket

Nosso objetivo:

- Revisão rápida sobre Socket
- Programação de Sockets TCP e UDP com Java
- Desenvolver um servidor Web

Interface de transporte de dados

- Os processos enviam e recebem mensagens através da interface de *Sockets*, uma API do sistema de operação
- Um *socket* permite a comunicação extremo a extremo



API *Sockets* (Application Programming Interface)

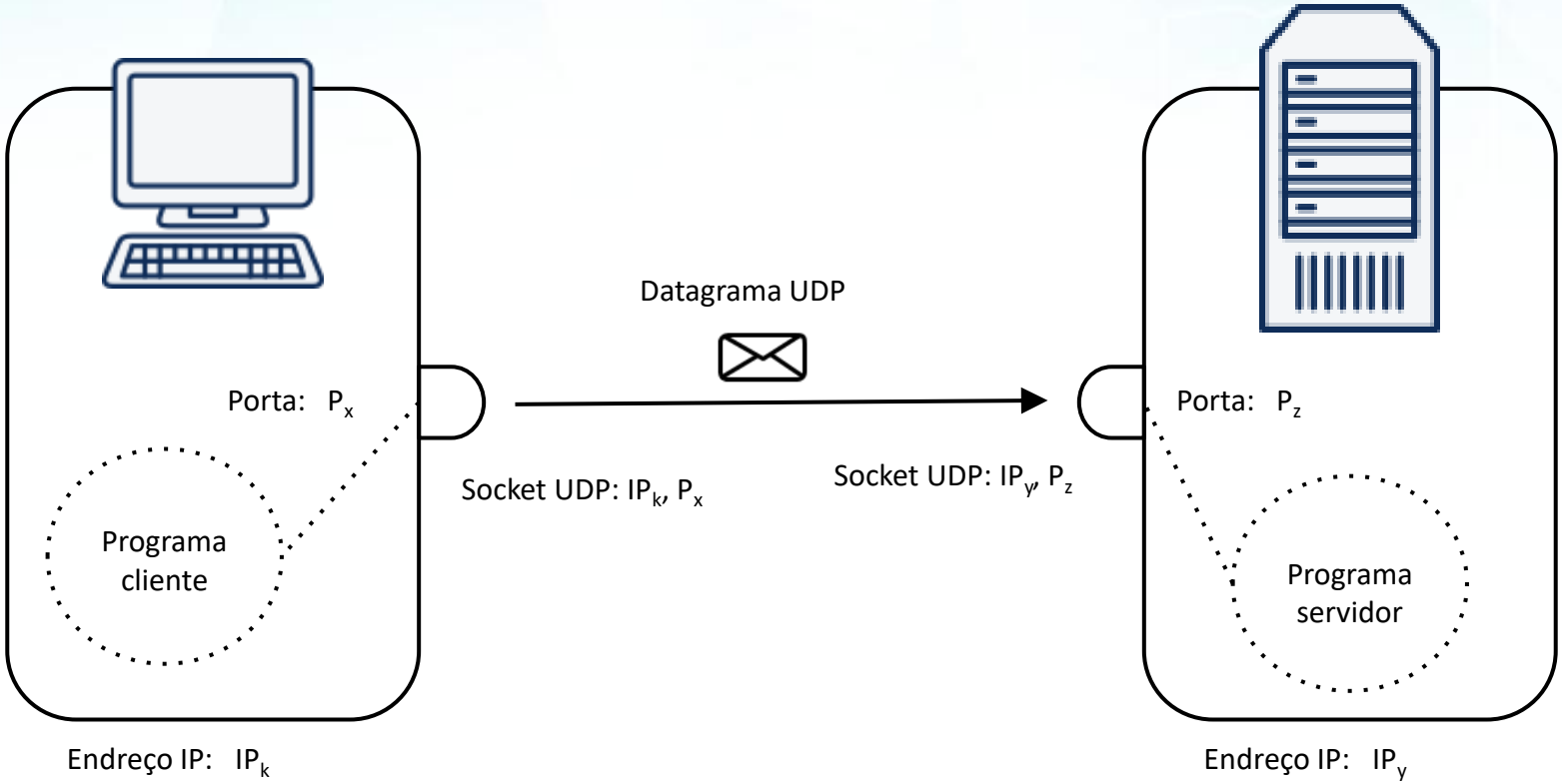
- Fornece primitivas para criar *sockets*, enviar e receber mensagens processadas e formatadas pelas aplicações
- Exemplo: Java *Socket API*
  - Class Socket
  - Métodos: *socket(), send(), write(), receive(), read(), close(), ...*
- O sistema de operação disponibiliza assim um serviço de comunicação generalizada implementado sobre o serviço de rede IP

Transportes e suas características

- **TCP**
  - Orientado conexão (exige conexão prévia)
  - Transporte fiável

- **UDP**
  - Muito simples de usar
  - Serviço para *datagramas* sem conexão (não exige conexão prévia)
  - Tem as mesmas garantias de banda, latência, controlo de fluxo ou saturação que o nível IP (“melhor esforço”)

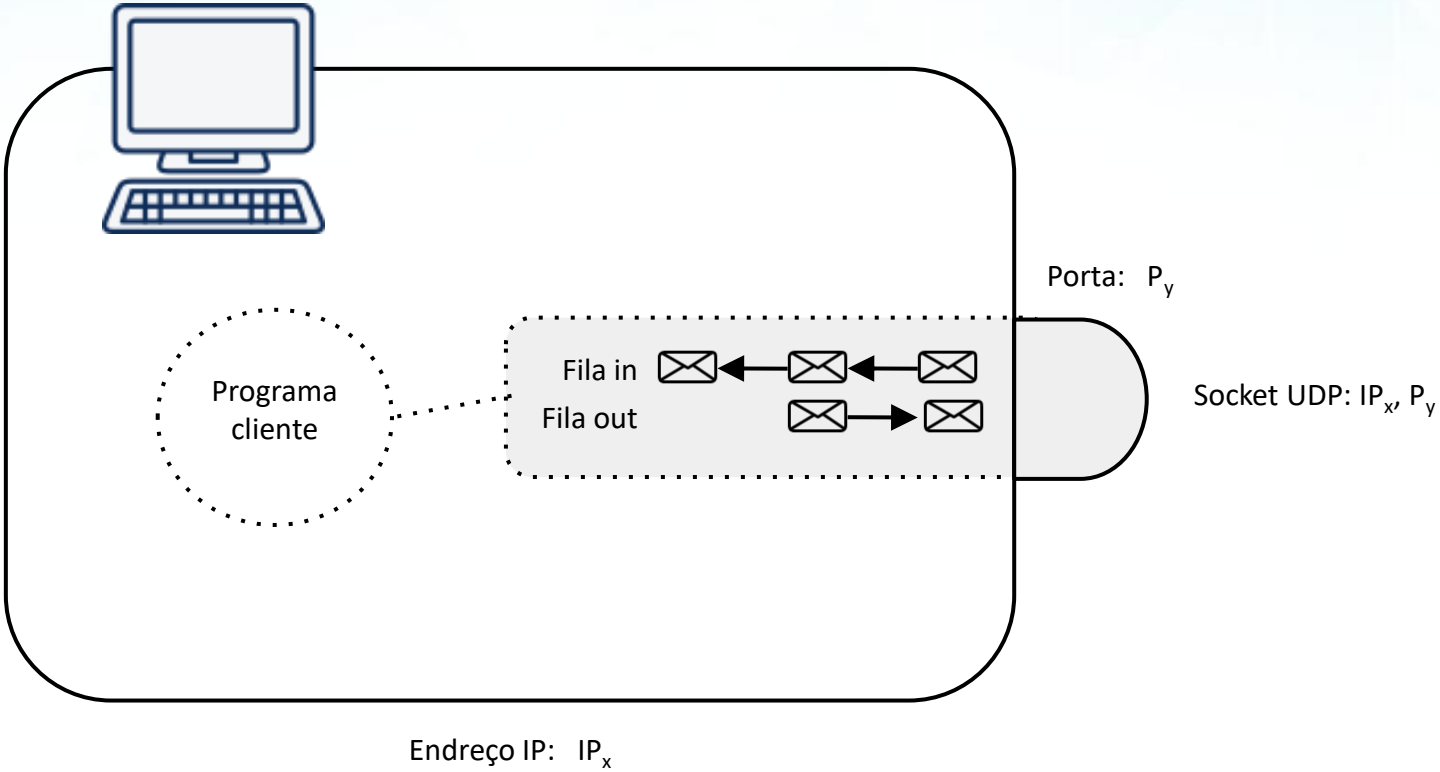
Comunicação com Sockets UDP



Class InetAddress

```
InetAddress myself =
      InetAddress.getByName("localhost");

InetAddress myself =
      InetAddress.getByName("127.0.0.1");

InetAddress myself = InetAddress.getLocalHost();

InetAddress server =
      InetAddress.getByName("www.wikipedia.org");

InetAddress server = InetAddress.getByName("200.10.78.9");

InetAddress [ ] servers =
      InetAddress.getAllByName("google.com");
```

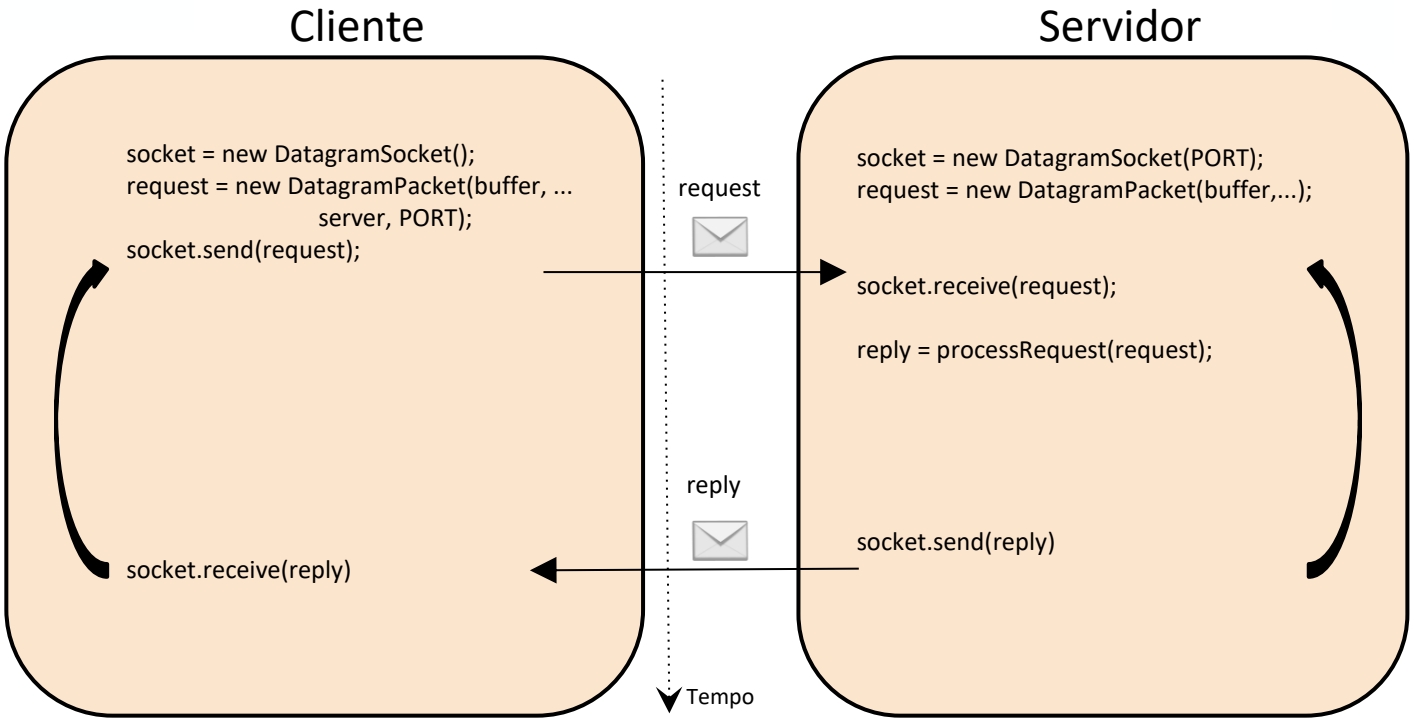
Sockets UDP


Porta: P_y
Programa cliente
Fila in
Fila out
Socket UDP: IP_x, P_y
Endreço IP: IP_x

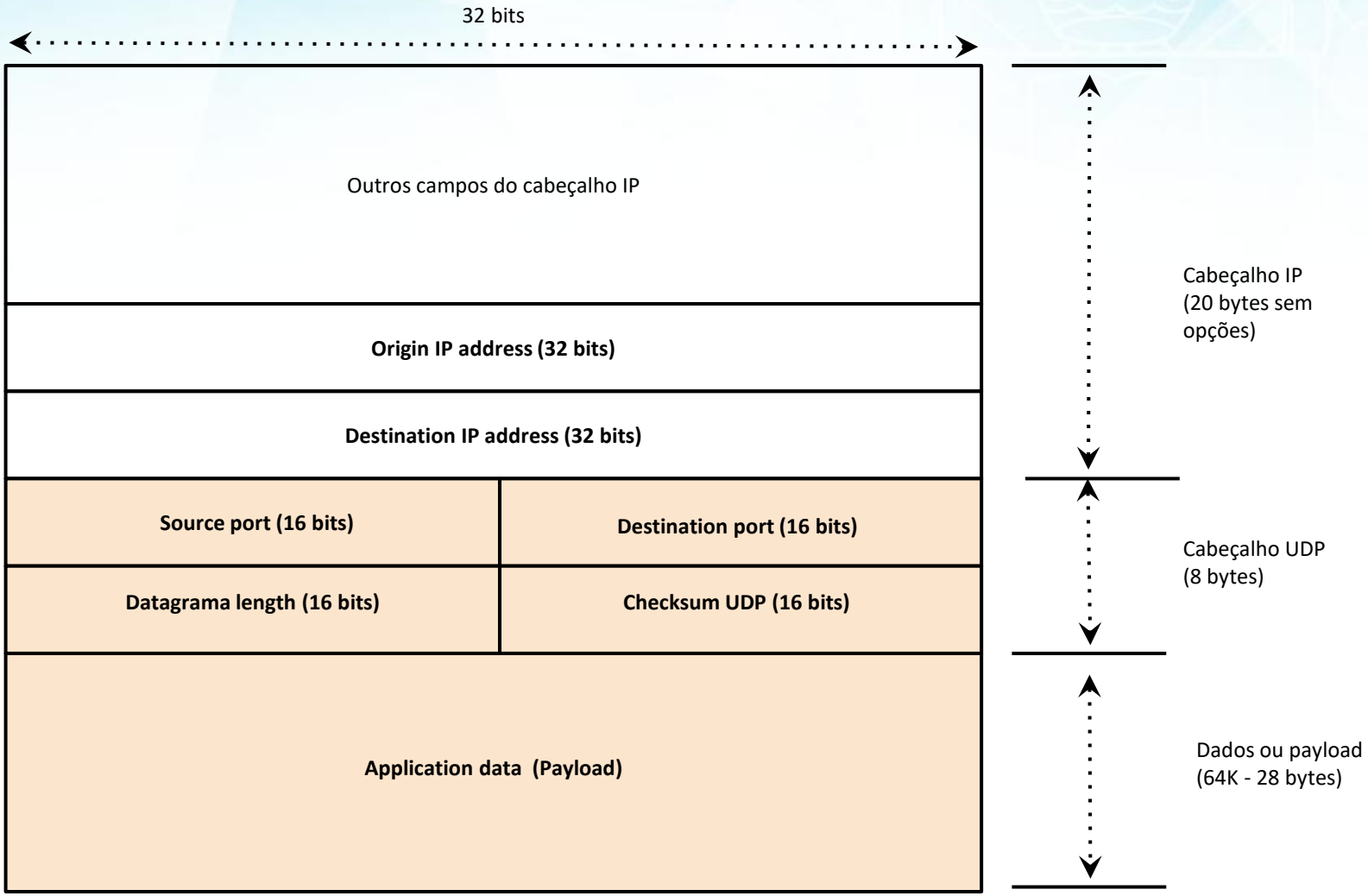
Class DatagramSocket

```
DatagramSocket clientSocket = new DatagramSocket();
```

```
DatagramSocket serverSocket = new DatagramSocket(PORT);
```



Formato dos *datagramas* UDP



Class DatagramPacket

```
Scanner in = new Scanner( System.in );

System.out.printf("Type a message: ");

String msg = in.nextLine();

byte[] msgData = msg.getBytes();

DatagramPacket request =  new DatagramPacket(msgData,
      msgData.length, server, port);

socket.send(request);
```

Receção de um DatagramPacket

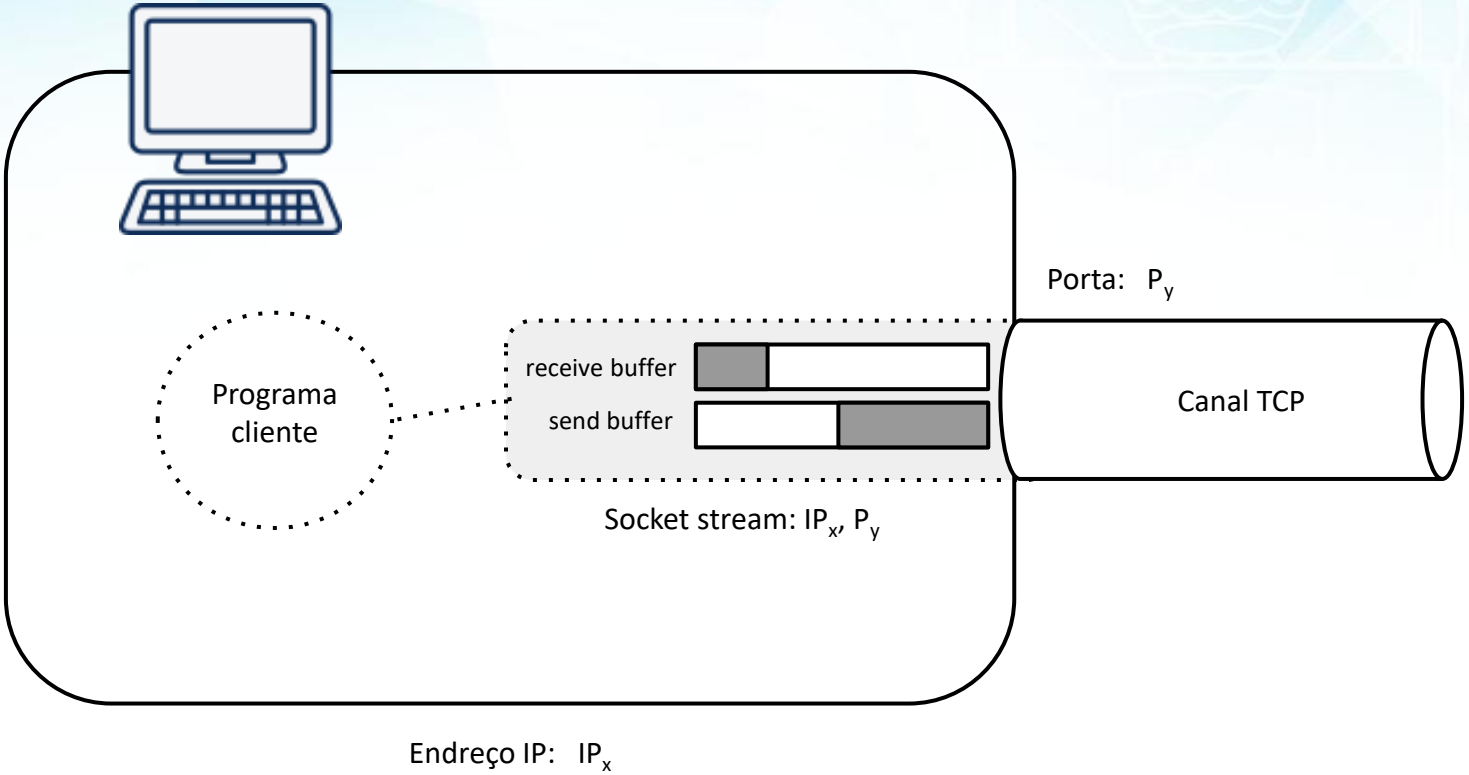
```
byte[] buffer = new byte[MAXMSG];

DatagramPacket reply = new DatagramPacket(buffer,
       buffer.length);

socket.receive(reply);
```

Exemplo Completo “UDP Echo Client”

```
import java.net.*;
import java.util.*;

public class UDPEchoClient {
   private static final int MAXMSG = 255;
   private static final int PORT = 8000;
   public static void main(String[] args) throws Exception {
      if (args.length != 1) {
         System.err.println("usage: java UDPEchoClient servidor");
         System.exit(0);
      }

      InetAddress serverAddress = InetAddress.getByName(args[0]);
      DatagramSocket socket = new DatagramSocket();
      Scanner in = new Scanner(System.in);
```

Continuação

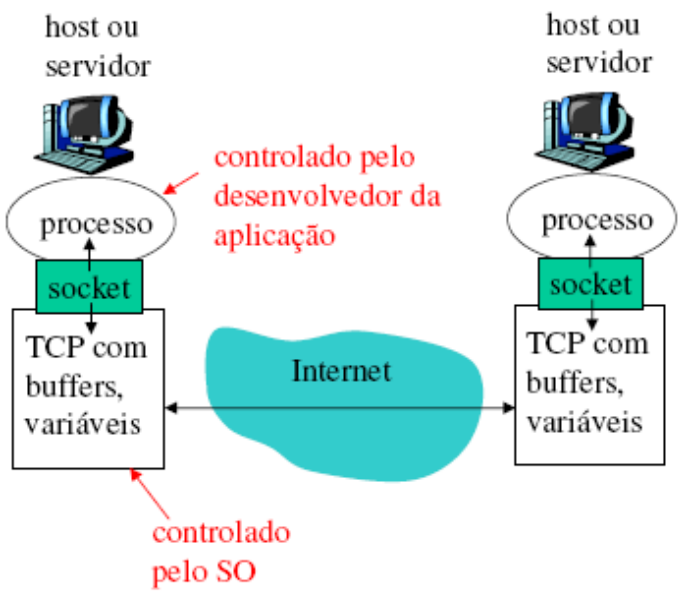
```
        for (;;) { // loop for ever
            System.out.printf("Type a message: ");
            String msg = in.nextLine();
            byte[] msgData = msg.getBytes();
            DatagramPacket request =
            new DatagramPacket(msgData, msgData.length,
                      serverAddress, PORT);
            socket.send(request);
            byte[] buffer = new byte[MAXMSG];
            DatagramPacket reply =
                      new DatagramPacket(buffer, MAXMSG);
            socket.receive(reply);
            String answer = new String(reply.getData(),0,
                             reply.getLength());
            System.out.printf("The answer is: \"%s\"\n", answer);
        } // never reached, leaves by exception
    // socket.close();
  }
}
```


Porta: P_y
Programa cliente
receive buffer
send buffer
Canal TCP
Socket stream: IP_x, P_y
Endreço IP: IP_x

Comunicação entre processos

- Processos em hosts distintos comunicam-se por meio de envio de mensagens...
- enviadas e recebidas através de seu socket

Socket é a interface entre a camada de aplicação e a de transporte



Programação de Socket TCP - Client

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class TCPclient {

    public static void main(String[] args) throws Exception  {
     //lendo do teclado
    String inFromUser = new Scanner(System.in).next();
    //criando um socket TCP
    Socket sock = new Socket("localhost", 2000);
    //stream de saida
    DataOutputStream socketOut = new DataOutputStream(sock.getOutputStream());
    socketOut.writeBytes(inFromUser + '\n');

    //resposta do servidor
    BufferedReader socketIn = new BufferedReader(new
    InputStreamReader(sock.getInputStream()));
    System.out.println(socketIn.readLine());
            }
    }
```

Programação de Socket TCP - Client

```
import java.io.*;
import java.net.*;

public class TCPserver {

    public static void main(String argv[]) throws Exception {
        String inFromClient;
        String outToClient;
        //socket de "boas vindas"
        ServerSocket welcomeSocket = new ServerSocket(2000);

        while(true) {
        //socket de conexão TCP
        Socket sock = welcomeSocket.accept();

        //buffer de entrada, que recebe um stream
        BufferedReader socketIn = new BufferedReader(new InputStreamReader(sock.getInputStream()));
        inFromClient = socketIn.readLine();
        outToClient = inFromClient.toUpperCase() + '\n';
        //stream de saida
        DataOutputStream socketOut = new DataOutputStream(sock.getOutputStream());//stream de saida
        //escrevendo no socket
        socketOut.writeBytes(outToClient);
        sock.close();
        }
    }
}
```

Exercício

- Faça um “Hello [endereço IP do servidor]” e retorne do servidor um “HELLO [endereço IP do cliente]”

- OBS: O cliente deve fechar a conexão após receber a resposta do servidor ou dar um timeout de 30 segundos.

Programação de Socket UDP - Client

```
import java.net.*;
import java.util.Scanner;

class UDPclient {

    public static void main(String args[]) throws Exception {
        String inFromUser = new Scanner(System.in).next() + '\n'; //entrada do usuário
        DatagramSocket clientSocket = new DatagramSocket(); //socket UDP
        InetAddress IPServer = InetAddress.getByName("localhost"); //IP do servidor

        byte[] sendData = new byte[1024]; //dados a serem enviados
        sendData = inFromUser.getBytes();
        //criando o pacote de envio
    DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPServer, 5000);
        clientSocket.send(sendPacket);

        byte[] receiveData = new byte[1024]; //dados recebidos
        DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);

        clientSocket.receive(receivePacket);    //recebendo o pacote
        String inFromServer = new String(receivePacket.getData());
        System.out.println("FROM SERVER: " + inFromServer);
        clientSocket.close();
    }
}
```

Programação de Socket UDP - Server

```
import java.net.*;
class UDPserver {

	public static void main(String args[]) throws Exception {
	DatagramSocket serverSocket = new DatagramSocket(5000);
	byte[] receiveData = new byte[1024] ; byte[] sendData = new byte[1024];
	String inFromClient, outToClient;
	InetAddress clientIP; int port;
	while(true) {
		//pacote a ser recebido
	DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
	serverSocket.receive(receivePacket);  //recebendo o pacotes
	inFromClient = new String(receivePacket.getData()); //colocando na string os dados recebidos
	clientIP = receivePacket.getAddress(); //pegando o IP e porta do pacote que chegou
	port = receivePacket.getPort();

	outToClient = inFromClient.toUpperCase();
	sendData = outToClient.getBytes();
		//enviando pacote de resposta
	DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, clientIP, port);
	serverSocket.send(sendPacket);
	}
	}
}
```

Exercício

- Faça, por meio de UDP, o cliente enviar dois números e o servidor responder com a subtração deles.

- OBS: O cliente deve encerrar após receber a resposta do servidor ou dar um timeout de 30 segundos.

Exercíco: Explique o que faz a aplicação a seguir

```
import java.io.IOException;
import java.net.Socket;
import java.net.UnknownHostException;

public class PortScan {
        public static String endereco = "localhost";
        public static void main(String[] args) {
              StringBuffer portasOk, portasNok;
              portasOk = new StringBuffer();
              portasNok = new StringBuffer();
              Socket s;
             for (int i = 70 ; i <= 100; i++){
                try {
                      s = new Socket(endereco, i);
                      portasOk.append(i + " ");
                      System.out.println("Porta " + i + " Aberta");
                      s.close();
                 } catch (UnknownHostException h){
                      System.out.println(h.getMessage());
                      System.exit(0);
                 } catch (IOException e) {
                      portasNok.append(i + " ");
                      System.out.println("Porta " + i + " Fechada");
                 }
            }//for
            System.out.println("Portas abertas: " +
portasOk.toString() + "\nPortas
fechadas: " + portasNok.toString());
      }//main
}
```

Exercícios

- Suponha um servidor UDP aceitando pacotes na porta 20001. É possível que mais de um computador envie pacotes UDP para tal host e tal porta? Por quê?

- Suponha um servidor TCP com uma conexão já aberta com um host clienteA. É possível a qualquer outro host cliente mandar dados ao servidor usando a conexão do clienteA já aberta? Por quê?